PS-5: GPU kernel, CPU emulation and digest parity

Date 2026-10-18. **Status:** **the device run did not happen and is BLOCKED (no GPU, no owner-approved allocation or cap).** No GPU figure, speedup or ratio appears anywhere in this report. Every executed number is a **CPU emulation** number or a **compile-time** number of the pinned `nvcc` 13.3.73. Protocol: `crypto research/pdp_oracle_speedup_20261011/protocols/PS-5.md`; this repository's `experiments/pdp-oracle-speedup-20261011/ps5/README.md` is the full record.

What was built

PS-3's straight-line program (hash-frozen, `pdp-slp/1`) is turned by `pdp_ps5_emit` into one statement per gate; every gate is one `lop3` (the fused xor-and is one ternary operation). The same generated statements are compiled by `nvcc` as the CUDA kernel (never run) and by a C compiler as the CPU emulation, which a parity driver runs over the PS-0 corpus.

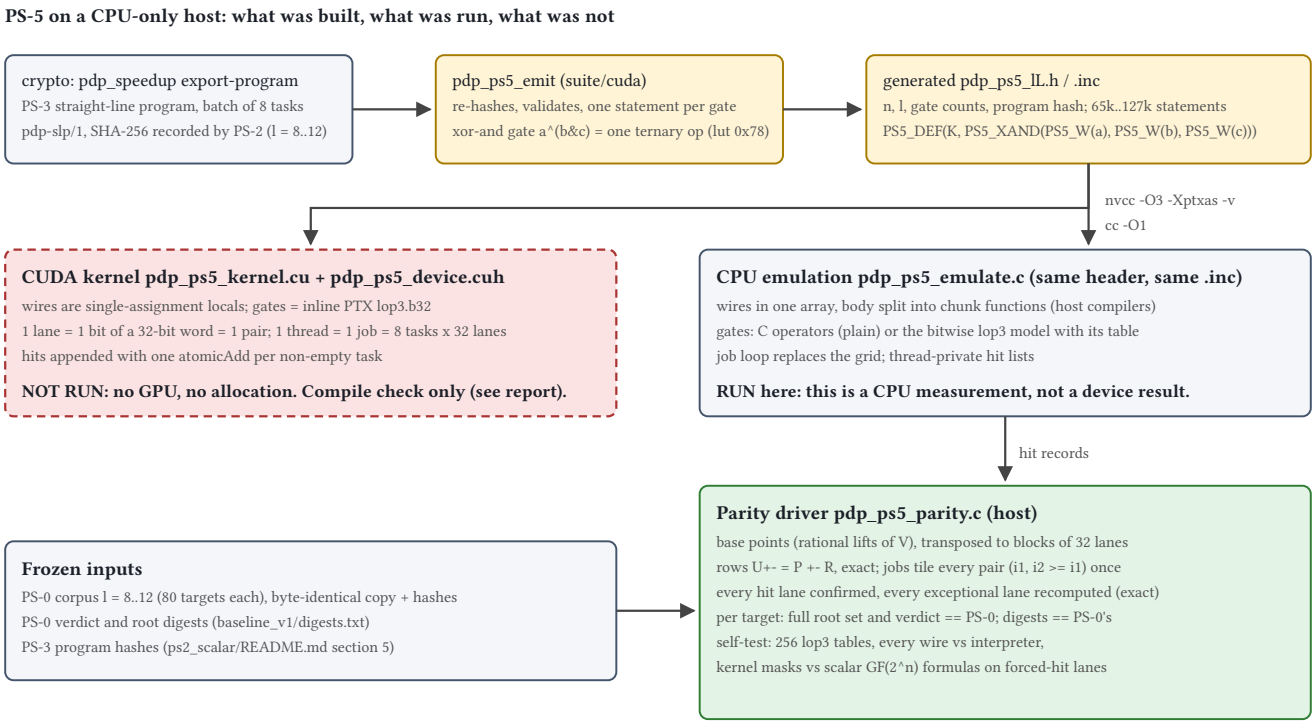


Figure 1: Pipeline and what was run. Source: `figures/ps5_pipeline.svg`.

Requirement status (summary; the full table is in the README)

Requirement	Status
Frozen program by hash, l = 8..12	verified complete: the five hashes equal PS-2's; fresh export byte-identical
CUDA kernel, `lop3`, one pair per lane, generated	implemented; compile-checked for sm_80 (l = 8: PTX has exactly 63,630 `lop3.b32` = the program's gates); **never run on a device**
CPU emulation of the kernel's arithmetic	implemented and verified on the CPU (same generated code; storage binding and operator form named in the README)
Replay PS-0 corpus, identical digests and root sets, l = 8..12	verified complete on the CPU: 211,265,840 pairs, 400 targets, 0 mismatches
Same gate on the device	**BLOCKED**: no GPU, no allocation
Counted bit operations per pair	verified complete (derived, cross-checked against the PTX); table below
Pairs per second on a device; A/A; ABAB vs PS-3	**BLOCKED**
>= 50x PS-3 (success) / < 20x (stop)	**not evaluated** (no device); requirement open
Expected ratio from a datasheet and microbenchmark	**not filled: no device, no datasheet-sourced figure**
l = 13..16	l = 13 optional sample only; 14..16 not attempted
`fes_gpu.rs` bridge (paired PR)	not attempted; `fes_gpu.rs` untouched

Parity with PS-0 (CPU emulation, 80 targets per rung)

l	n	base pts	pairs/target	pairs replayed	hit lanes conf./ unconf.	exc. lanes	roots (targets)	root-set / verdict mismatches	verdict digest = PS-0	root digest = PS-0
8	24	128	8,256	660,480	96 / 0	0	32 (25)	0 / 0	yes	yes
9	27	253	32,131	2,570,480	105 / 0	0	35 (28)	0 / 0	yes	yes
10	30	464	107,880	8,630,400	90 / 0	0	30 (25)	0 / 0	yes	yes
11	33	1,007	507,528	40,602,240	78 / 0	0	26 (25)	0 / 0	yes	yes
12	36	1,992	1,985,028	158,802,240	90 / 0	0	30 (23)	0 / 0	yes	yes

all				211,265,840	459 / 0	0		0 / 0	5 of 5	5 of 5

Not run in this report.

Not run.

Self-test (CPU emulation, 50 repetitions per rung)

l	lop3 tables	wires compared / mismatches	lanes compared / mismatches	reference hit lanes	reference exceptional lanes
8	pass	2,073,024 / 0	409,300 / 0	51,329	25,611
9	pass	2,508,416 / 0	409,300 / 0	51,322	25,611
10	pass	2,789,056 / 0	409,300 / 0	51,323	25,611
11	pass	3,347,648 / 0	409,300 / 0	51,318	25,611
12	pass	4,051,648 / 0	409,300 / 0	51,318	25,611

Counted bit operations per pair

Per lane slot: bit operations = G x 32 / (8 tasks x 32 lanes) = G / 8 (fused: xand = 1; two-input: xand = 2). Per valid pair: divide by the lane utilisation u = valid pairs / lane slots of this job layout.

l	G (lop3/run)	xand	fused / lane slot	two-input / lane slot	u	fused / valid pair	two-input / valid pair	lop3 / valid pair
8	63,630	46,360	7,953.75	13,748.75	0.2520	31,568.4	54,568.7	986.51
9	77,092	57,816	9,636.50	16,863.50	0.4961	19,424.8	33,992.6	607.02
10	85,718	70,296	10,714.75	19,501.75	0.6125	17,493.2	31,839.1	546.66
11	103,030	85,536	12,878.75	23,570.75	0.7796	16,519.6	30,234.3	516.24
12	124,886	107,320	15,610.75	29,025.75	0.8677	17,990.4	33,450.3	562.20

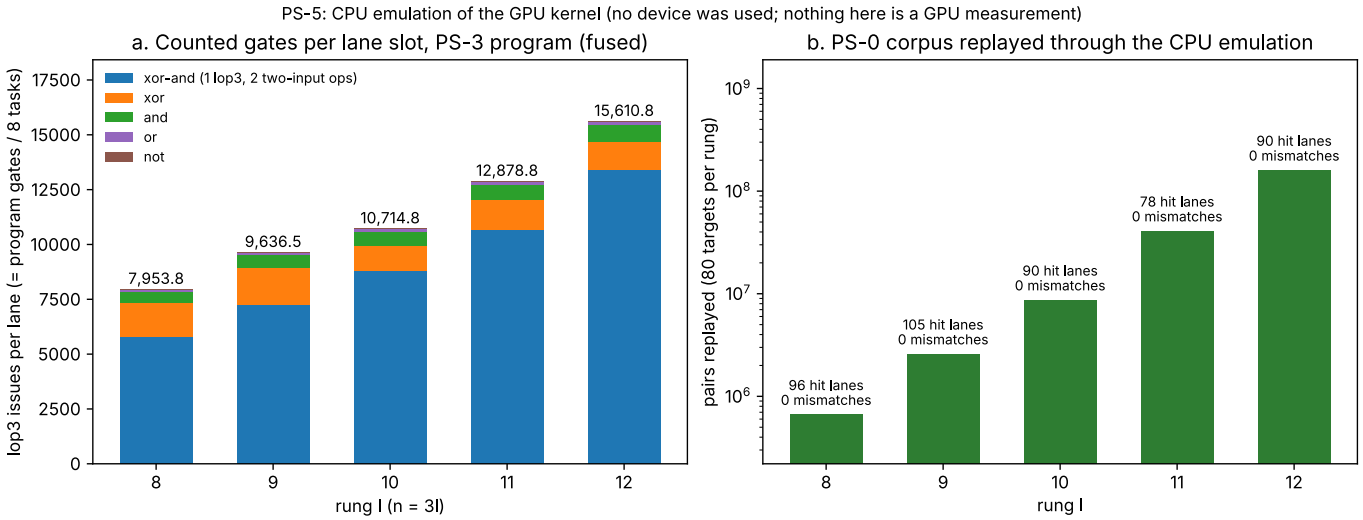


Figure 2: Left: counted gates per lane slot by kind (n = 8 tasks per program). Right: pairs replayed through the CPU emulation per rung (80 targets). Source: `raw/emit`, `raw/parity_l*.jsonl`; `figures/ps5_ops_and_coverage.svg`.

Compile evidence, l = 8 (sm_80, nvcc 13.3.73 -O3; nothing run)

ptxas: 255 registers, 4584 bytes stack, 9724 bytes spill stores, 14396 bytes spill loads. SASS (static): 70976 instructions, of which LOP3 63752 (tables 0x78: 46389, 0x3c: 12269, 0xc0: 4142, 0xfc: 887); LDL 3599, STL 2431, LDG 390, STG 32, ATOMG 8. The program has 46,360 xand, 12,334 xor, 4,008 and, 880 or, 48 not gates. These are static compiler outputs, not performance.

Gaps and open requirements

The device run and everything that needs it (pairs per second, A/A, ABAB against PS-3, the 50x and 20x thresholds, the expected ratio) stay open. Also: the Gray-position mapping is moot and was not built; the emulation keeps wires in an array and runs operators proved equal to the `lop3` tables (hardware `lop3` semantics not verified here); the job layout wastes lanes (u = 0.25 to 0.87); the l = 8 kernel needs 255 registers and spills; the PS-0 corpus has no exceptional lanes; no Rust bridge, NVRTC path or CI job; l = 14..16 not attempted. Host: Intel(R) Xeon(R) Processor @ 2.10GHz, 4 vCPU, kernel 6.18.44-fc-v139, shared; GPU devices: none.